



28TECH
Become A Better Developer

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG





Hướng đối tượng - Object Oriented Programming hay OOP là một mô hình lập trình quan trọng, được ứng dụng rộng rãi trong phát triển phần mềm. OOP tập trung xoay quanh dữ liệu và đối tượng thay vì tập trung vào thủ tục và hàm như các mô hình lập trình truyền thống.



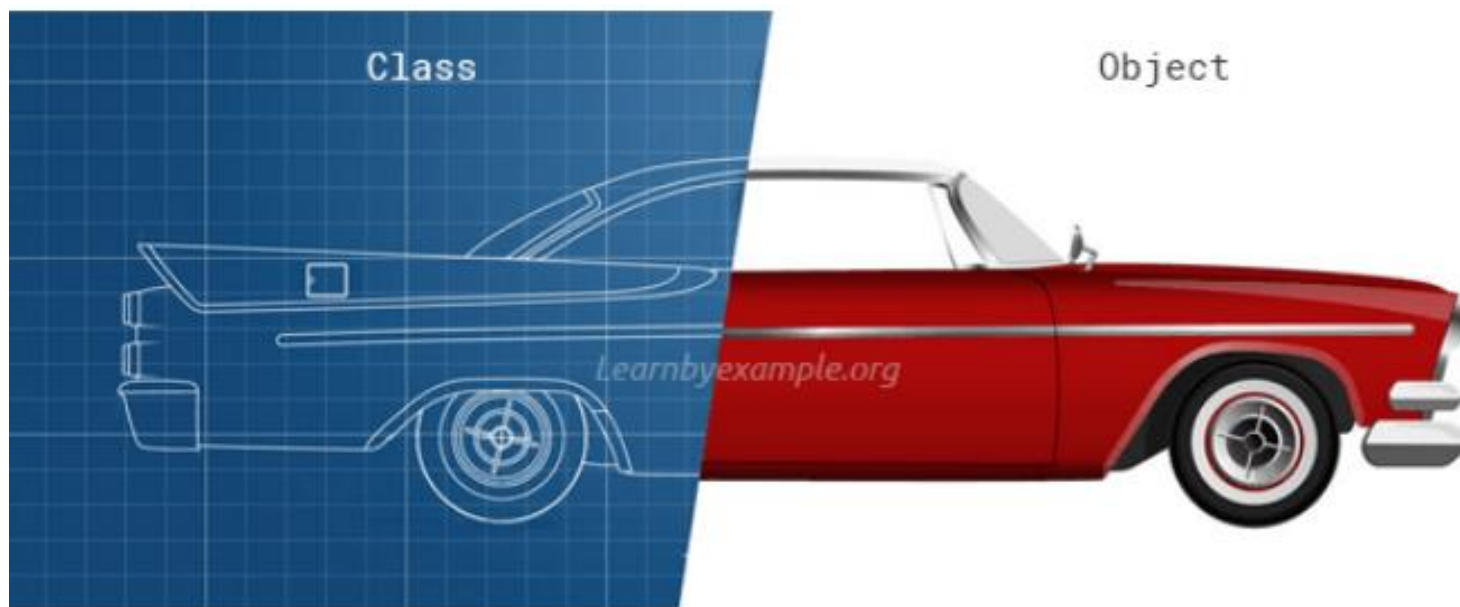
1. Class và Object:



Hai khái niệm quan trọng của lập trình hướng đối tượng là : **Lớp (Class)** và **Đối tượng (Object)**.

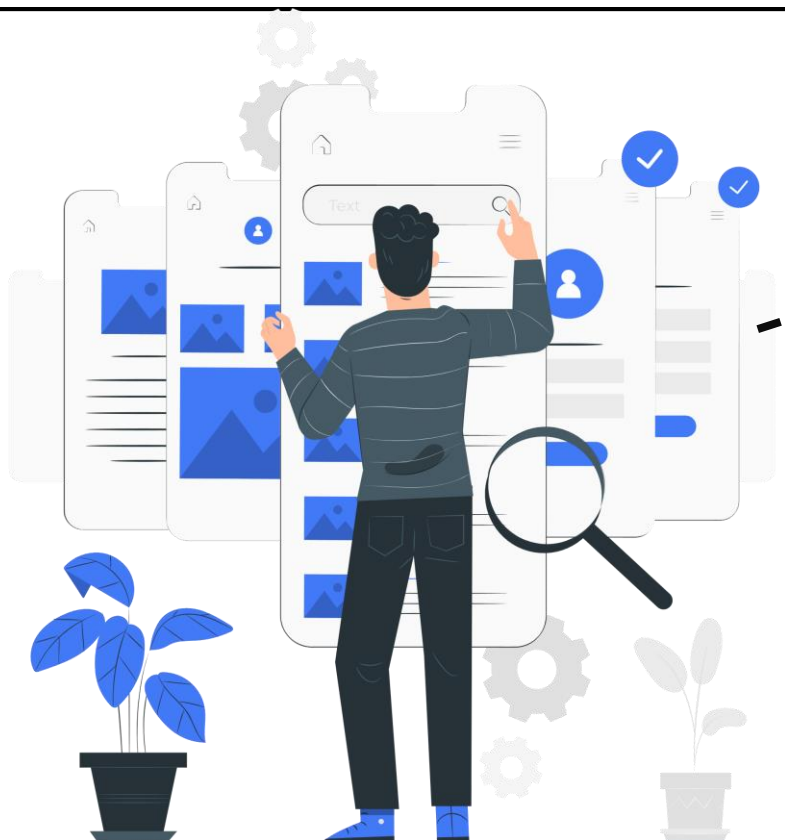


Mục tiêu của OOP đó là cố gắng đưa các thực thể, đối tượng trong thực tế vào phần mềm.



(Hình minh họa)

1. Class và Object:



Ví dụ phần mềm của bạn cần quản lý những đối tượng như Người, Xe, Sách thì khi đó các đối tượng này cũng được xây dựng trong phần mềm của bạn dưới dạng các lớp.



Có thể hiểu đơn giản, lớp chính là bản mô phỏng của một đối tượng trong thực tế.

1. Class và Object:

Lớp có nhiệm vụ phác họa, mô phỏng những thông tin chung còn khái niệm về đối tượng của lớp lại là một thực thể cụ thể của lớp đó.



Ví dụ: Bạn cần xây dựng một lớp là Person để mô phỏng con người thì khi đó một đối tượng của lớp Person là “Nguyễn Văn Tèo” chính là một thực thể, một đối tượng của lớp Person.

2. Xây dựng Class:

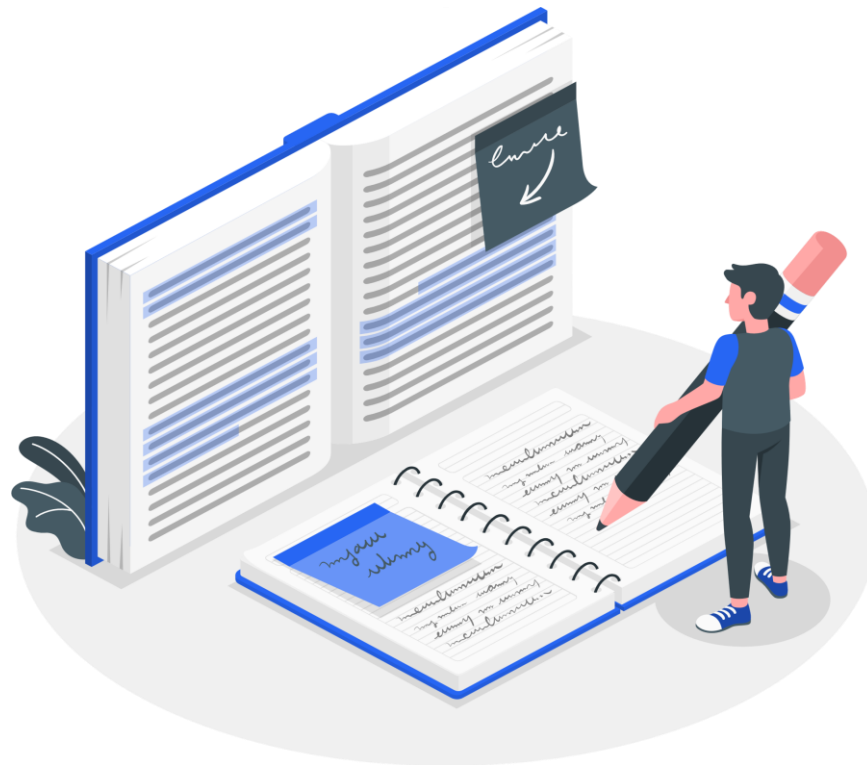


Để xây dựng lớp ta sử dụng từ khóa class theo sau bởi tên class. Ví dụ về cách định nghĩa lớp Person. Câu lệnh pass được đặt trong lớp Person để chỉ ra lớp Person rỗng.

```
class Person:  
    pass
```



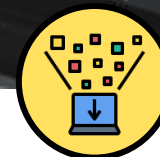
Cú pháp để **tạo nên một đối tượng(instance)** của lớp Person: `p = Person()`.



3. Phương thức `__init()` và tham số `self`:



`__init()` là một phương thức đặc biệt của lớp, phương thức này sẽ tự động được gọi mỗi khi bạn tạo ra một đối tượng của lớp.



Thông thường phương thức này **đóng vai trò như hàm tạo (constructor)** ở các ngôn ngữ lập trình như C++, Java để có thể khởi tạo những thông tin ban đầu cho đối tượng.

3. Phương thức `__init__()` và tham số `self`:



Tham số `self` đề cập đến chính đối tượng hiện tại, `self` luôn được khai báo như là tham số đầu tiên của các phương thức trong lớp như một tiêu chuẩn.



Phương thức `__init__()` này có tham số đầu tiên là `self`. Nó được sử dụng để truy vấn hoặc thiết lập các thuộc tính của đối tượng cụ thể

EXAMPLE

```
class Person:
    def __init__(self):
        print('Ham init duoc goi !')

if __name__ == '__main__':
    p = Person()
```

OUTPUT

Ham init duoc goi !

4. Thuộc tính và phương thức của lớp:

a. Thuộc tính:



Thuộc tính là những thứ riêng lẻ để phân biệt một đối tượng này với một đối tượng khác.



Chúng xác định vẻ ngoài, trạng thái hoặc các phẩm chất khác của đối tượng đó. Để truy cập vào thuộc tính của lớp ta sử dụng toán tử '.'



Các thuộc tính được định nghĩa trong các lớp bởi các biến và mỗi đối tượng có thể có các giá trị riêng cho các biến đó.



Có hai loại thuộc tính: Instance attributes và Class attributes.

Ví dụ:

Đối với class Person sẽ có những thuộc tính như :

- name: 28tech, nam, long,...
- job: dev, CEO, Engineer,...
- address: Ha Noi, HCM, Bac Ninh,...

4. Thuộc tính và phương thức của lớp:

a. Thuộc tính:

EXAMPLE

```
class Person:
    #class attribute
    nationality = 'Viet Nam'
    #instance attribute
    def __init__(self, name, job):
        self.name = name
        self.job = job

if __name__ == '__main__':
    p1 = Person('28tech', 'dev')
    p2 = Person('Nam', 'CEO')
    print(p1.nationality, p1.name, p1.job)
    print(p2.nationality, p2.name, p2.job)
```

OUTPUT

```
Viet Nam 28tech dev
Viet Nam Nam CEO
```

4. Thuộc tính và phương thức của lớp:

b. Phương thức:



Phương thức mô tả các hành động của đối tượng. Để truy cập vào phương thức của lớp ta sử dụng toán tử '.'



Phương thức cũng tồn tại phương thức thuộc về lớp và phương thức thuộc về đối tượng.

Ví dụ:

Đối với class Person sẽ có những phương thức như :

- Đi lại
- Giao tiếp
- Ăn uống
- Học tập

EXAMPLE

```
class Person:
    #class attribute
    nationality = 'Viet Nam'
    #instance attribute
    def __init__(self, name, job):
        self.name = name
        self.job = job
    def eat(self):
        print('Eat')

if __name__ == '__main__':
    p1 = Person('28tech', 'dev')
    p1.eat()
```

OUTPUT

Eat

5. Tính chất đóng gói:



Trong Python không tồn tại các Access Modifier như Public, Private và Protected mà ta sử dụng dấu gạch dưới để chỉ định access modifier cho các thuộc tính trong lớp.

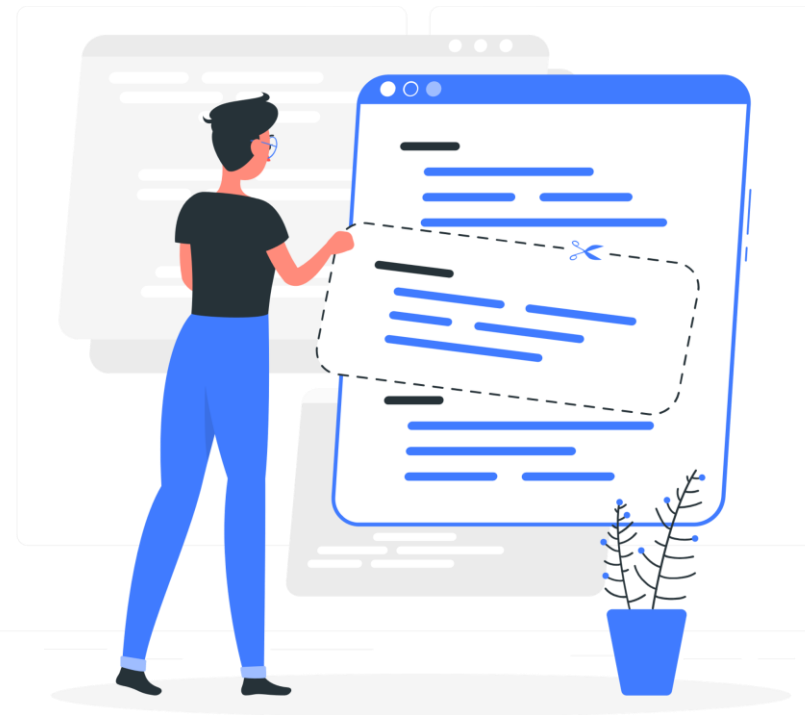


Nếu thuộc tính không có dấu gạch dưới đi trước thì là public, một dấu gạch dưới trước tên là protected và 2 dấu gạch dưới trước tên là private.

Public: Truy cập được bên trong và bên ngoài lớp.

Private: Chỉ truy cập được bên trong lớp.

Protected: Truy cập được bên trong lớp và ở các lớp con.



5. Tính chất đóng gói:

Ví dụ:

EXAMPLE

```
class Person:
    def __init__(self, name, job, salary):
        self.name = name #public
        self._job = job #protected
        self.__salary = salary #private
    def showInfor(self):
        print('Name : {}, Job : {}, Salary : {}'.format(self.name, self._job, self.__salary))

if __name__ == '__main__':
    p = Person('28tech', 'dev', '1000')
    p.showInfor()
```

OUTPUT

Name : 28tech, Job : dev, Salary : 1000

5. Tính chất đóng gói:

Ví dụ về thuộc tính public

EXAMPLE

```
class Person:
    def __init__(self, name):
        self.name = name #public

if __name__ == '__main__':
    p = Person('28tech')
    print(p.name)
```

OUTPUT

28tech

Ví dụ về thuộc tính private

EXAMPLE

```
class Person:
    def __init__(self, name):
        self.__name = name #private

if __name__ == '__main__':
    p = Person('28tech')
    print(p.__name)
```

OUTPUT

AttributeError: 'Person' object
has no attribute '__name'